Praneesh R V
CB.SC.U4CYS23036

1,

Code:

```
use std::fs::{File, OpenOptions};
use std::io::{self, BufRead, BufReader, Write};
use std::path::Path;
use std::fmt;

#[derive(Debug, Clone, PartialEq)]
struct Item {
   id: u32,
   name: String,
   price: f32,
   quantity: u32,
}

#[derive(Debug, PartialEq)]
enum OrderStatus {
   Completed(Item),
   OutOfStock,
   ItemNotFound,
   InvalidOrder,
}

#[derive(Debug)]
enum AppError {
   IoError(io::Error),
   ParseError(String),
   MissingFile(String),
```

```rust
}

impl fmt::Display for AppError {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        match *self {
            AppError::IoError(ref err) => write!(f, "IO Error: {}", err),
            AppError::ParseError(ref err) => write!(f, "Parse Error: {}", err),
            AppError::MissingFile(ref filename) => write!(f, "Missing File: {}", filename),
        }
    }
}

impl From<io::Error> for AppError {
    fn from(err: io::Error) -> AppError {
        AppError::IoError(err)
    }
}

fn read_inventory(filename: &str) -> Result<Vec<Item>, AppError>
{
    if !Path::new(filename).exists() {
        return Err(AppError::MissingFile(filename.to_string()));
    }

    let file = File::open(filename)?;
    let reader = BufReader::new(file);
    let mut inventory = Vec::new();

    for (index, line) in reader.lines().enumerate() {
        let line = line?;
        let parts: Vec<&str> = line.split(',').collect();
```

```
        if parts.len() != 4 {
            log_error(format!("Line {}: Invalid format in
inventory file: {}", index + 1, line))?;
            continue;
        }

        let id = parts[0].trim().parse::<u32>().map_err(|_|
AppError::ParseError(format!("Line {}: Invalid ID", index +
1)))?;
        let name = parts[1].trim().to_string();
        let price = parts[2].trim().parse::<f32>().map_err(|_|
AppError::ParseError(format!("Line {}: Invalid Price", index +
1)))?;
        let quantity = parts[3].trim().parse::<u32>().map_err(|_|
AppError::ParseError(format!("Line {}: Invalid Quantity", index
+ 1)))?;

        inventory.push(Item { id, name, price, quantity });
    }

    Ok(inventory)
}

fn save_inventory(filename: &str, inventory: &Vec<Item>) ->
Result<(), AppError> {
    let mut file = File::create(filename)?;
    for item in inventory {
        writeln!(file, "{},{},{},{}", item.id, item.name,
item.price, item.quantity)?;
    }
    Ok(())
}

fn log_error(message: String) -> Result<(), AppError> {
```

```rust
    let mut file = OpenOptions::new()
        .create(true)
        .append(true)
        .open("errors.log")?;
    writeln!(file, "{}", message)?;
    Ok(())
}


fn process_one_line(inventory: &mut Vec<Item>, line: &str) ->
Result<OrderStatus, AppError> {
    let parts: Vec<&str> = line.split(',').collect();
    if parts.len() != 2 {
        return Ok(OrderStatus::InvalidOrder);
    }

    let item_id = match parts[0].trim().parse::<u32>() {
        Ok(id) => id,
        Err(_) => return Ok(OrderStatus::InvalidOrder),
    };

    let qty_needed = match parts[1].trim().parse::<u32>() {
        Ok(qty) => qty,
        Err(_) => return Ok(OrderStatus::InvalidOrder),
    };

    if let Some(item) = inventory.iter_mut().find(|i| i.id ==
item_id) {
        if item.quantity >= qty_needed {
            item.quantity -= qty_needed;
            let mut bought_item = item.clone();
            bought_item.quantity = qty_needed;
            Ok(OrderStatus::Completed(bought_item))
        } else {
```

```
            Ok(OrderStatus::OutOfStock)
        }
    } else {
        Ok(OrderStatus::ItemNotFound)
    }
}

fn process_orders(inventory: &mut Vec<Item>, orders_file: &str)
-> Result<(), AppError> {
    if !Path::new(orders_file).exists() {
        return
Err(AppError::MissingFile(orders_file.to_string()));
    }

    let file = File::open(orders_file)?;
    let reader = BufReader::new(file);
    let mut bill_file = File::create("bill.txt")?;

    writeln!(bill_file, "{:<20} {:<10} {:<10} {:<10}", "Item
Name", "Qty", "Cost", "Total")?;
    writeln!(bill_file, "{}", "-".repeat(55))?;

    let mut grand_total = 0.0;

    for (index, line) in reader.lines().enumerate() {
        let line_content = line?;

        match process_one_line(inventory, &line_content)? {
            OrderStatus::Completed(item) => {
                let cost = item.price * item.quantity as f32;
                grand_total += cost;
                writeln!(bill_file, "{:<20} {:<10} {:.2}
{:.2}", item.name, item.quantity, item.price, cost)?;
```

```
                println!("Order for item {} (qty: {}) processed
successfully.", item.id, item.quantity);
            },
            OrderStatus::OutOfStock => {
                let _ = log_error(format!("Order Line {}: Out of
Stock - {}", index + 1, line_content));
                println!("Order failed: Out of Stock.");
            },
            OrderStatus::ItemNotFound => {
                let _ = log_error(format!("Order Line {}: Item
Not Found - {}", index + 1, line_content));
                println!("Order failed: Item Not Found.");
            },
            OrderStatus::InvalidOrder => {
                let _ = log_error(format!("Line {}: Invalid
order format - {}", index + 1, line_content));
            }
        }
    }

    writeln!(bill_file, "{}", "-".repeat(55))?;
    writeln!(bill_file, "{:<43} {:.2}", "Grand Total:",
grand_total)?;

    Ok(())
}

fn main() {
    match read_inventory("inventory.txt") {
        Ok(mut inventory) => {
            println!("Inventory loaded successfully.");
            match process_orders(&mut inventory, "orders.txt") {
                Ok(_) => {
```

```
                    println!("Orders processed. Check bill.txt
for details.");
                    if let Err(e) =
save_inventory("inventory.txt", &inventory) {
                        eprintln!("Failed to save updated
inventory: {}", e);
                    } else {
                        println!("Inventory updated and saved to
inventory.txt.");
                    }
                },
                Err(e) => eprintln!("Error processing orders:
{}", e),
            }
        },
        Err(e) => eprintln!("Error loading inventory: {}", e),
    }
}
```

Inventory.txt
201,Pen,10.5,40
202,Notebook,25,26
203,Eraser,5,100
204,Pencil,3,0

Orders.txt
201,5
202,2

203,110
204,1
205,1
201,test

Errors.log

```
Order Line 3: Out of Stock - 203,110
Order Line 4: Out of Stock - 204,1
Order Line 5: Item Not Found - 205,1
Line 6: Invalid order format - 201,test
Order Line 3: Out of Stock - 203,110
Order Line 4: Out of Stock - 204,1
Order Line 5: Item Not Found - 205,1
Line 6: Invalid order format - 201,test
```

Bill.txt

Item Name	Qty	Cost	Total
Pen	5	10.50	52.50
Notebook	2	25.00	50.00
Grand Total:			102.50

```
praneesh@ShadowEternity ~/Praneesh/Academics/Sem6/POPL/Rust/Lab-9 ‹main●›
$ ./1
Inventory loaded successfully.
Order for item 201 (qty: 5) processed successfully.
Order for item 202 (qty: 2) processed successfully.
Order failed: Out of Stock.
Order failed: Out of Stock.
Order failed: Item Not Found.
Orders processed. Check bill.txt for details.
Inventory updated and saved to inventory.txt.
praneesh@ShadowEternity ~/Praneesh/Academics/Sem6/POPL/Rust/Lab-9 ‹main●›
$
```

2,

Code:

```rust
use std::collections::{HashMap, HashSet};
use std::fs::File;
use std::io::{self, BufRead, BufReader, Write};
use std::str::FromStr;

#[derive(Debug, PartialEq, Eq, Hash, Clone)]
enum ActionType {
    LOGIN,
    DOWNLOAD,
    UPLOAD,
}

#[derive(Debug, PartialEq, Eq, Clone)]
enum StatusType {
    SUCCESS,
    FAIL,
}

#[derive(Debug)]
struct LogEntry {
    timestamp: String,
```

```rust
    ip: String,
    action: ActionType,
    status: StatusType,
}

#[derive(Debug)]
enum LogError {
    Io(io::Error),
    InvalidFormat(String),
    UnknownAction(String),
    UnknownStatus(String),
}

impl From<io::Error> for LogError {
    fn from(err: io::Error) -> Self {
        LogError::Io(err)
    }
}

impl FromStr for ActionType {
    type Err = LogError;
    fn from_str(s: &str) -> Result<Self, Self::Err> {
        match s {
            "LOGIN" => Ok(ActionType::LOGIN),
            "DOWNLOAD" => Ok(ActionType::DOWNLOAD),
            "UPLOAD" => Ok(ActionType::UPLOAD),
            _ => Err(LogError::UnknownAction(s.to_string())),
        }
    }
}

impl FromStr for StatusType {
    type Err = LogError;
    fn from_str(s: &str) -> Result<Self, Self::Err> {
```

```
        match s {
            "SUCCESS" => Ok(StatusType::SUCCESS),
            "FAIL" => Ok(StatusType::FAIL),
            _ => Err(LogError::UnknownStatus(s.to_string())),
        }
    }
}

fn parse_line(line: &str) -> Result<LogEntry, LogError> {
    let parts: Vec<&str> = line.split('|').collect();
    if parts.len() != 4 {
        return Err(LogError::InvalidFormat(format!("Line should
have 4 parts separated by '|', found {}", parts.len())));
    }

    let timestamp = parts[0].to_string();
    let ip = parts[1].to_string();
    let action = ActionType::from_str(parts[2])?;
    let status = StatusType::from_str(parts[3])?;

    Ok(LogEntry {
        timestamp,
        ip,
        action,
        status,
    })
}

fn main() -> io::Result<()> {
    let input_path = "access.log";
    let corrupted_path = "corrupted_lines.txt";
    let report_path = "threat_report.txt";

    let file = File::open(input_path)?;
```

```
    let reader = BufReader::new(file);

    let mut corrupted_file = File::create(corrupted_path)?;
    let mut entries: Vec<LogEntry> = Vec::new();

    for line_result in reader.lines() {
        let line = line_result?;
        match parse_line(&line) {
            Ok(entry) => entries.push(entry),
            Err(e) => {
                let err_msg = match e {
                    LogError::Io(err) => format!("IO Error: {}",
err),
                    LogError::InvalidFormat(msg) =>
format!("Invalid Format: {}", msg),
                    LogError::UnknownAction(act) =>
format!("Unknown Action: {}", act),
                    LogError::UnknownStatus(stat) =>
format!("Unknown Status: {}", stat),
                };
                writeln!(corrupted_file, "Line: '{}' | Error:
{}", line, err_msg)?;
            }
        }
    }

    // Analysis
    let mut login_failures: HashMap<String, i32> =
HashMap::new();
    let mut total_failures: HashMap<String, i32> =
HashMap::new();
    let mut ip_actions: HashMap<String, HashSet<ActionType>> =
HashMap::new();
```

```
    for entry in &entries {
        // Track total failures for reporting context
        if entry.status == StatusType::FAIL {
            *total_failures.entry(entry.ip.clone()).or_insert(0)
+= 1;
        }

        // Track specific login failures
        if entry.action == ActionType::LOGIN && entry.status ==
StatusType::FAIL {
            *login_failures.entry(entry.ip.clone()).or_insert(0)
+= 1;
        }

        // Track actions for suspicious pattern detection
        ip_actions
            .entry(entry.ip.clone())
            .or_insert_with(HashSet::new)
            .insert(entry.action.clone());
    }

    let mut report_file = File::create(report_path)?;
    writeln!(report_file, "Suspicious Activity Report")?;
    writeln!(report_file, "==========================")?;

    // Collect all unique IPs encountered
    let all_ips: HashSet<&String> =
ip_actions.keys().chain(login_failures.keys()).collect();
    let mut reported_ips: HashSet<&String> = HashSet::new(); //
to avoid duplicates if analyzed differently

    for ip in all_ips {
        if reported_ips.contains(ip) {
            continue;
```

```
        }

        let mut reasons = Vec::new();
        let fail_count = *total_failures.get(ip).unwrap_or(&0);
        let login_fail_count =
*login_failures.get(ip).unwrap_or(&0);

        // Check: > 3 failed logins
        if login_fail_count > 3 {
            reasons.push(format!("Excessive failed logins ({})",
login_fail_count));
        }

        // Check: UPLOAD and DOWNLOAD in same session
        if let Some(actions) = ip_actions.get(ip) {
            if actions.contains(&ActionType::UPLOAD) &&
actions.contains(&ActionType::DOWNLOAD) {
                reasons.push("Performed both UPLOAD and
DOWNLOAD".to_string());
            }
        }

        if !reasons.is_empty() {
            writeln!(
                report_file,
                "IP: {} | Failures: {} | Reason: {}",
                ip,
                fail_count,
                reasons.join(", ")
            )?;
            reported_ips.insert(ip);
        }
    }
```

```
    println!("Analysis complete. Check {} and {}",
corrupted_path, report_path);
    Ok(())
}
```

```
┌─praneesh@ShadowEternity ~/Praneesh/Academics/Sem6/POPL/Rust/Lab-9/2 ‹main•›
└─$ ./2
Analysis complete. Check corrupted_lines.txt and threat_report.txt
┌─praneesh@ShadowEternity ~/Praneesh/Academics/Sem6/POPL/Rust/Lab-9/2 ‹main•›
└─$
```

Access.log

```
2026-02-18 18:30|192.168.1.5|LOGIN|FAIL
2026-02-18 18:31|192.168.1.5|LOGIN|FAIL
2026-02-18 18:32|192.168.1.5|LOGIN|FAIL
2026-02-18 18:33|192.168.1.5|LOGIN|FAIL
2026-02-18 18:35|10.0.0.1|LOGIN|SUCCESS
2026-02-18 18:36|10.0.0.1|UPLOAD|SUCCESS
2026-02-18 18:40|10.0.0.1|DOWNLOAD|SUCCESS
2026-02-18 18:45|172.16.0.5|LOGIN|SUCCESS
2026-02-18 18:46|172.16.0.5|DOWNLOAD|SUCCESS
bad_format_line
2026-02-18 18:50|192.168.1.9|INVALID_ACTION|fail
2026-02-18 18:55|192.168.1.10|LOGIN|UNKNOWN_STATUS
2026-02-18 19:00|192.168.1.20|LOGIN|FAIL
2026-02-18 19:01|192.168.1.20|LOGIN|FAIL
```

Corrupted_lines.txt

Line: 'bad_format_line' | Error: Invalid Format: Line should have 4 parts separated by '|', found 1

Line: '2026-02-18 18:50|192.168.1.9|INVALID_ACTION|fail' | Error: Unknown Action: INVALID_ACTION
Line: '2026-02-18 18:55|192.168.1.10|LOGIN|UNKNOWN_STATUS' | Error: Unknown Status: UNKNOWN_STATUS

threat_report.txt

Suspicious Activity Report
==========
IP: 192.168.1.5 | Failures: 4 | Reason: Excessive failed logins (4)
IP: 10.0.0.1 | Failures: 0 | Reason: Performed both UPLOAD and DOWNLOAD